



Revature Expense Manager – Phase 2



Brian Paul | PYTEST · ALLURE · RUNBOOK/JIRA

Matthew Landuyt | JMETER CONFIGURATIONS · LOAD DATABASE · LOAD REPORTS

Scot Putnam | FLASK API · JAVALIN API · CONTRACT SUITES

Matthew Washburn | REACT · SELENIUM · E2E

Landon Wu | JUNIT 5 · MOCKITO · JACOCO

Tech Stack

Application & Quality Engineering Architecture

Application Engineering Stack

FRONTEND & E2E TESTING

React: UI components and frontend application.

Cucumber / Selenium: BDD scenarios testing the full expense flow in-browser.

BACKEND SERVICES

Python / Flask: Handles the Employee API.

Java / Javalin: Handles the Manager API.

DATABASE & SECURITY

SQLite: Shared relational database.

JWT (JSON Web Token): Authentication and token handling across APIs.

Quality Engineering & Testing Stack

UNIT & INTEGRATION TESTING

PyTest / Requests: Automated testing for the Python/Flask API.

JUnit 5 + RESTAssured: Automated unit/integration testing for Java/Javalin API.

PERFORMANCE / LOAD TESTING

Apache JMeter

Stress, load, and performance testing on APIs.

REPORTING & TRACEABILITY

Allure

Test reporting framework to track execution, coverage, and defects.

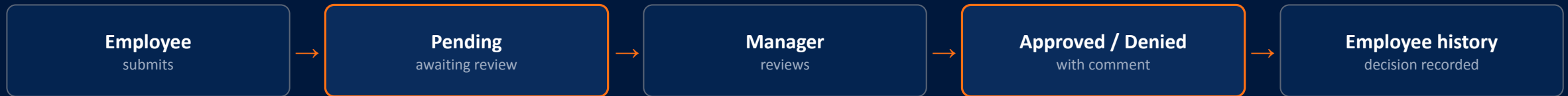
STLC | Team Contributions Across the Software Testing Lifecycle



Revature Expense Manager

Two apps, two roles, one shared database. One React front end over both.

EXPENSE LIFECYCLE



Employee App

PYTHON · FLASK

- Submit expenses with amount, category, and date
- Edit or delete an expense while it is still pending
- Track status and review full decision history

Manager App

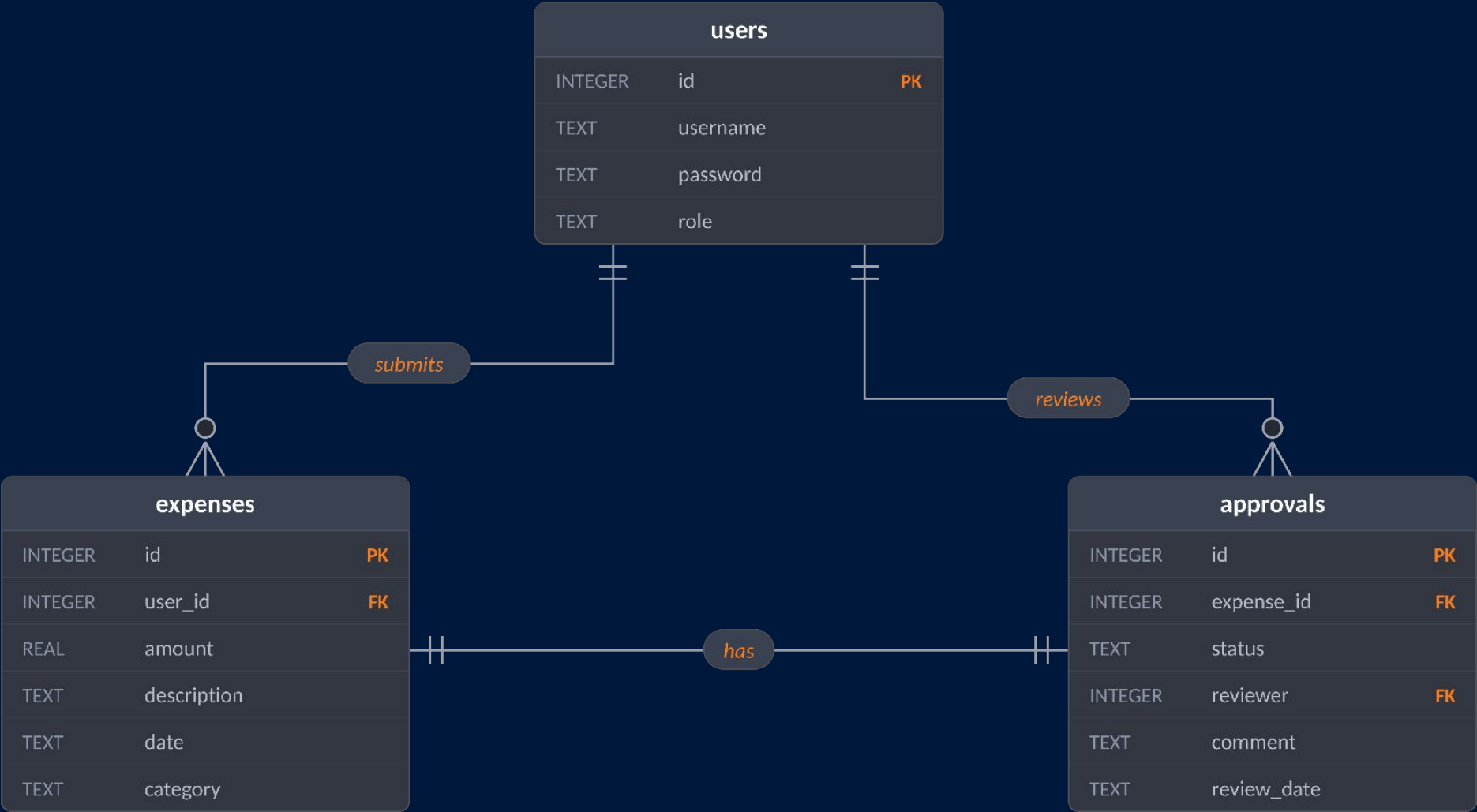
JAVA · JAVALIN

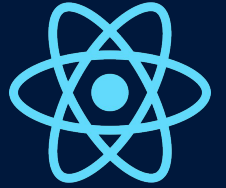
- Work a queue of all pending submissions
- Approve or deny with a comment back to the employee
- Report by employee, category, or date range, with CSV export

One React front end · Shared SQLite database · JWT authentication · bcrypt password hashing

Database Schema

SQLite ERD · users, expenses, approvals





Front End, Built for Testing

Every screen you are about to see was designed to be automatable.



Single-page routing

Moving between dashboard, pending, and history never reloads the page.



A data-testid on every interactable

Stable test ids live on the shared components, and row actions are scoped by record id. Tests target an expense, never a table position.



Protected routes, JWT-backed

Sessions are verified server-side. A direct URL with no valid session redirects straight to the welcome page.



Blocker: no way to verify a session

Neither backend could confirm a live session, so I added an auth/me endpoint to both. ProtectedRoute revalidates on every mount.

End-to-End Testing

10 scenarios · 5 feature files

Real browser, real APIs, real database. Nothing mocked.

Gherkin .feature



Selenium steps

EMPLOYEE LIFECYCLE

HAPPY PATH

- Submit → confirm pending → edit → delete

SAD PATHS

- Invalid date rejected, corrected resubmit succeeds
- Wrong password rejected with a clear error
- Manager credentials blocked at employee login
- Direct dashboard URL redirects to sign-in

MANAGER LIFECYCLE

HAPPY PATHS

- Review queue → approve with comment
- Review queue → deny with comment

SAD PATHS

- Employee credentials blocked at manager login
- Direct dashboard URL redirects to sign-in

CROSS-ROLE LIFECYCLE

HAPPY PATH

- Employee submits → manager approves → decision and comment appear in the employee ledger

ONE BROWSER SESSION

- Shared driver and context carry the run across both roles



Explicit waits, never sleeps



Wait for text, not just visibility



One driver, shared across roles

Clear specialists. Shared visibility.

OWNERSHIP: Stable ownership kept delivery focused.

PHASE 2: GOALS AND ROLES

1. Stabilize the core app flows and wire the main UI to the APIs.
2. Raise automated test coverage across Java and Python layers.
3. Validate API behavior and load/performance foundations.
4. Deliver E2E workflow coverage and execution visibility.



- Matthew W. - 🎨 Frontend Dev/E2E QAE :: 1, 4
- Landon - ☕ Java QAE :: 2
- Brian - 🦆🏆 Python QA + Release QAE :: 2,4
- Scot - 🦆☕ Backend Contract QAE + Bug Owner :: 1, 3
- Matthew L. - 📊 JMeter / API QAE :: 3

VISIBILITY & TRACEABILITY: Jira kept every lane visible.

PHASE 2: SPRINT LIGHTWEIGHT JIRA KANBAN

	Work	Assignee	Reporter
☐	> ☒ KAN-11 Allure Integration	BP Brian Paul	BP Brian Paul
☐	> ☒ KAN-16 Employee Login	BP Brian Paul	BP Brian Paul
☐	> ☒ KAN-17 Employee Submit Expense	BP Brian Paul	BP Brian Paul
☐	☒ KAN-32 UI Implementation	MW Matthew Washburn	BP Brian Paul
☐	☒ KAN-33 Foundation Unit Tests	BP Brian Paul	BP Brian Paul
☐	☒ KAN-34 Gap Edge-Case Unit Tests	BP Brian Paul	BP Brian Paul
☐	☒ KAN-35 API Testing	BP Brian Paul	BP Brian Paul
☐	☒ KAN-36 E2E Testing	MW Matthew Washburn	BP Brian Paul
☐	> ☒ KAN-18 Employee View Ledger	BP Brian Paul	BP Brian Paul
☐	> ☒ KAN-19 Employee Edit Delete	BP Brian Paul	BP Brian Paul
☐	> ☒ KAN-20 Manager Login	L Landon	BP Brian Paul
☐	> ☒ KAN-21 Manager View Pending Expenses	L Landon	BP Brian Paul
☐	> ☒ KAN-22 Manager Approve Deny	L Landon	BP Brian Paul
☐	> ☒ KAN-23 Manager Reports	S Scot	BP Brian Paul
☐	✗ KAN-87 [manager_app] POST /login returns 500 inste...	S Scot	S Scot
☐	✗ KAN-99 [manager_app] - PUT /expenses/{id}/review ...	S Scot	S Scot
☐	✗ KAN-91 [employee_app] - POST /auth/login (employe...	S Scot	S Scot
☐	☒ KAN-25 Global Database Seeding	ML Matt Landuyt	BP Brian Paul
☐	☒ KAN-24 Global JMeter Performance Testing Setup	ML Matt Landuyt	BP Brian Paul
☐	✗ KAN-14 [employee_app] Unexpected SQLite integrity ...	BP Brian Paul	BP Brian Paul
☐	✗ KAN-77 [manager_app] - POST /login returns 500 inst...	L Landon	L Landon
☐	✗ KAN-78 [manager_app] - PUT /expenses/{id}/review r...	L Landon	L Landon
☐	✗ KAN-76 [manager_app] - GET /employees returns em...	S Scot	S Scot

- Progress across:
 - User story testing completion
 - User Interface
 - System testing
 - Allure
- Bug reporting
- Shared test requirements documentation

A testing pyramid built around application behavior

API JOURNEYS

2 focused workflows

live Flask · JWT cookie

INTEGRATION

11 Repository + SQLite

temp_db · schema · persistence

UNIT

61 Controller + service

pytest-mock · Flask test client

74

initial baseline

114

current passing

91%

branch coverage

2

live journeys

COVERAGE

74 passed, 40 deselected in 4.55s

Name	Stmts	Miss	Branch	BrPart	Cover
employee_app/api/auth.py	22	1	6	0	96%
employee_app/api/auth_controller.py	40	4	4	2	86%
employee_app/api/expense_controller.py	73	17	0	0	77%
employee_app/config/app_config.py	19	3	2	1	81%
employee_app/repository/database.py	28	1	4	1	94%
employee_app/repository/expense_model.py	10	0	0	0	100%
employee_app/repository/expense_repository.py	48	0	2	0	100%
employee_app/repository/ledger_entry_model.py	13	0	0	0	100%
employee_app/repository/user_model.py	15	3	6	3	71%
employee_app/repository/user_repository.py	29	0	4	0	100%
employee_app/service/authentication_service.py	49	7	12	3	84%
employee_app/service/expense_service.py	98	0	36	0	100%
TOTAL	444	36	76	10	91%

BEHAVIOR

```
@allure.epic("Employee Portal Backend")
@allure.feature("Expense Management")
@allure.parent_suite("Employee - API Layer")
@allure.suite("Employee API Journeys")
@allure.issue("KAN-16")
@allure.story("Employee Login and Authentication")
@allure.issue("KAN-17")
@allure.story("Submit New Expense")
class TestEmployeeApiJourneys:

    @allure.issue("KAN-18")
    @allure.story("View Expense Ledger")
    @allure.title("Happy path through submission, pending view, and ledger tracking")
    @allure.severity(allure.severity_level.CRITICAL)
    def test_employee_happy_path_submission_and_ledger_tracking(self, live_server):
        session = requests.Session()

        with allure.step("Step 1: Login and receive JWT cookie"):--
        with allure.step("Step 2: Submit a valid expense and capture expense_id"):--
        with allure.step("Step 3: View pending expenses and verify submitted expense is present"):--
        with allure.step("Step 4: View full ledger and verify contract includes pending and history arrays"):--
```

One result, three ways to ask a quality question

BUSINESS

What requirement did we prove?

- Epic
 - Feature
 - Story
 - KAN issue

ARCHITECTURE

Where did the test run?

- Parent suite
 - Suite
 - Test layer
 - Python / Java

RISK

What kind of confidence is it?

- smoke
 - negative
 - security
 - regression / edge case

BUSINESS VIEW

Behaviors

order name duration status

Status: 0 0 78 0 0 Marks:

Employee Portal Backend

78

Authentication

22

Employee Login and Authentication

22

Expense Management

56

Employee Login and Authentication

5

Modify and Delete Pending Expenses

24

Submit New Expense

18

View Expense Ledger

9

ARCHITECTURE VIEW

Suites

order name duration status

Status: 0 0 74 0 0 Marks:

Employee - API Layer

2

Employee API Journeys

2

Employee - Controller Layer

18

Authentication Controller

5

Expense Controller

13

Employee - Repository Layer

11

Expense Repository

6

User Repository

5

Employee - Service Layer

43

Authentication Service

12

Expense Service

31

RISK VIEW

Marker	Purpose	Example
unit	Service/controller logic with mocks	<code>.venv/bin/pytest -m unit</code>
db	Repository behavior against temporary SQLite	<code>.venv/bin/pytest -m db</code>
api	API and journey coverage	<code>.venv/bin/pytest -m api</code>
contract	Real HTTP, cookie/JWT, route wiring, and persistence-boundary contracts	<code>.venv/bin/pytest -m contract</code>
smoke	Critical happy paths	<code>.venv/bin/pytest -m smoke</code>
negative	Expected rejection and error behavior	<code>.venv/bin/pytest -m negative</code>
security	JWT, role, authorization, ownership, and disclosure behavior	<code>.venv/bin/pytest -m security</code>
regression	Fixed defects, including KAN-85	<code>.venv/bin/pytest -m regression</code>
edge_case	Boundary and minor behavior	<code>.venv/bin/pytest -m edge_case</code>



Unit Testing

Unit Testing – Manager App

Testing Strategy

- JUnit 5
- Mockito
- Mocked Dependencies
- Business Logic
- Happy & Sad Paths
- Confluence Backlog

Authentication

- Manager Login
- JWT Validation
- Authorization
- Invalid Credentials
- Missing/Invalid Tokens

Expense Review

- View Pending
- Approve Expenses
- Deny Expenses
- Manager Comments
- Invalid Review Requests

Reporting

- By Employee
- By Category
- By Date Range
- CSV Export
- Input Validation

Unit Testing – Value of Unit Testing



Regression Protection

- Detects issues early
- Protects core business logic
- Prevents existing functionality from breaking



Fast Feedback

- Fast, isolated execution
- No database or application startup required
- Supports rapid development



Testing Foundation

- **Unit Tests:** Business logic
- **API Tests:** Endpoint behavior & contracts
- **E2E Tests:** Complete user workflows



Integration Testing

Integration Testing + API Testing

- Unit Testing
 - mocked isolation – is the code written correctly?
- Integration Testing + API Testing
 - real server, real database, real HTTP
 - does the running service behave correctly?
- E2E
 - journey through the browser
- 97 integration tests · 55 Java · 42 Python

Allure Report 7/30/2026

14:53:07 – 14:54:07 (59s 783ms)

97

test cases

100%

Nothing is mocked

- Real server
 - Javalin and Flask boot on a live port
- Real database
 - rebuilt before every single test, from the same schema.sql that production uses
- Real auth
 - tokens signed by the production JwtService, tests an expired token too
- Two languages, same pattern

Suites		
order ↓↑ name ≡ duration ↓↑ status ↓↑		
Status:	0 0 97 0 0	Marks:
▼ Employee - API Layer		42
> Employee API Journeys		2
> Employee Authentication Contract		14
> Employee Expense Contract		15
> Employee Login Contract		11
▼ Manager - Integration Tests		55
> Authentication Contract		8
> Manager Expense Review		12
> Manager Login		7
> Manager Reports		16
> Viewing the Employee Roster		12

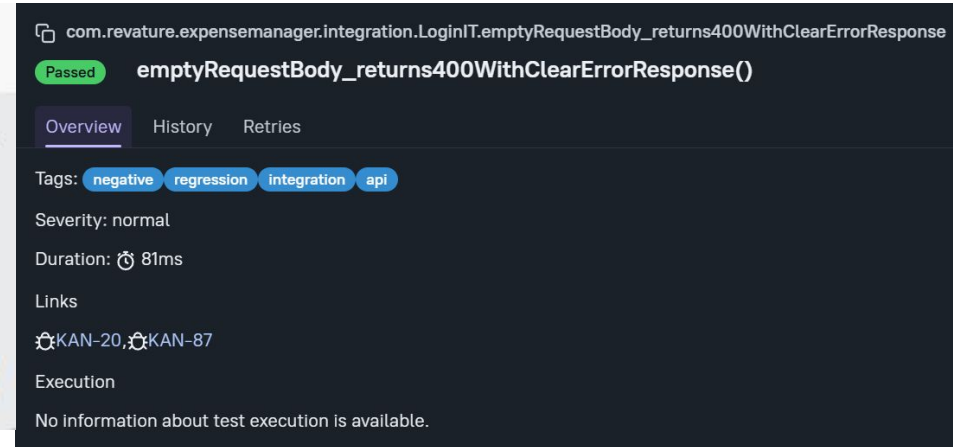
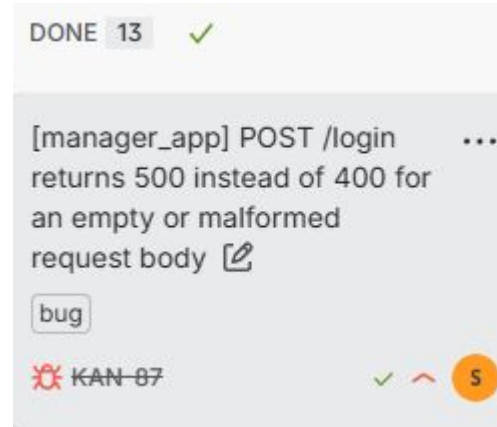
Testing requirements, as clickable filters

- Smoke
 - 15 Java · 4 Python
 - "Correct status codes for valid requests"
- Negative path
 - 38 Java · 33 Python
 - "Handle invalid input gracefully"
- Security
 - 19 Java · 23 Python
 - "Authentication and authorization are enforced"
- Edge case
 - 2 Java · 4 Python
 - Boundary behavior
- Regression
 - 4 Java · 5 Python

One defect. Found three times.

Defect: Empty or truncated JSON body → 500 Internal Server Error not 400

- KAN-87
 - Manager /login
- KAN-90
 - Manager /expenses/{id}/review
 - Same defect, different endpoint
- KAN-85
 - Employee /auth/login
 - Same test, different language.
- Invisible to unit tests; found at 90% coverage.
 - reported in Jira with bug template



Deterministic, and traceable to Jira

Why you can trust the suite

- What makes our tests deterministic?
 - Full DB rebuild per test
 - Tests on a random port
 - No shared state between tests
 - 0 flaky tests, 0 retries
- What makes our tests traceable?
 - Epic → Manager Portal Backend
 - Feature → Expense Approval
 - Story gg → Manager Approve Deny · KAN-22
- What a traceable and deterministic test allows:
 - A PO opens the report, finds their ticket, and sees what was proven about it.

Where that leaves us

- 97 integration + API tests, across two languages
- 0 failures
- 1 root cause defect – 3 endpoints, 2 languages, 3 regression tests
- 100% of API requirements traceable to a filter in the report



Performance Testing

Utilizing Jmeter test suite

Jmeter Results

- Utilizing Jmeter we have JMX paths and tests included for application performance management
- Testing paths include for both branches:
 - Write tests
 - Read tests
 - Login management
- Testing database setup to prevent database clutter
- All tests access API endpoints and test at scale
- Bottlenecks:
 - SQLite database limitations
 - Login hashing response times
- Jmeter includes automatic report HTML generation for ease of use and presentation;
 - Walk through test cases for writing small scale
 - Run test for basic login live for demonstration purposes



Retrospectives

Thank you!

Quality Ownership Across the Test Lifecycle

OWNER & LANE	RA REQUIREMENTS ANALYSIS	TP TEST PLANNING	TD CASE DEVELOPMENT	ES ENVIRONMENT SETUP	TE TEST EXECUTION	CC CYCLE CLOSURE	CONTRIBUTION & STACK
Brian Paul REPORTING ARCHITECTURE	RA	TP	TD	ES	TE	CC	Stood up the shared Allure reporting environment, authored the Allure strategy. the Python suite test plan/development/execution, and maintains the README runbook that makes every suite reproducible on any machine. PYTEST · ALLURE · RUNBOOK DOCUMENTATION · JIRA
Landon Wu UNIT & BRANCH COVERAGE	RA	TP	TD	ES	TE	CC	Branch-coverage analysis and test planning across the Java service and controller layers; unit case design for the authorization and approve/deny user stories, with defects logged back to the team. JUNIT 5 · MOCKITO · JACOCO · DEFECT REPORTS
Scot Putnam API CONTRACT & THROUGHPUT	RA	TP	TD	ES	TE	CC	Full-lifecycle ownership of contract testing across the employee and manager APIs — analysis, plan, fixture-backed environments, execution and defect reporting — plus the server-side optimization that cleared the hashing bottleneck. FLASK API · JAVALIN API · CONTRACT SUITES · FIXTURES
Matthew Washburn END-TO-END & UI AUTOMATION	RA	TP	TD	ES	TE	CC	Analysis, planning and Gherkin case design for the browser E2E layer; built the React frontend with stable test hooks so scenarios target intent, not brittle markup; reported a backend authorization defect and shipped the helper endpoint that fixed it. SELENIUM · CUCUMBER .FEATURE · REACT TEST HOOKS · ALLURE LABELS
Matthew Landuyt PERFORMANCE & LOAD	RA	TP	TD	ES	TE	CC	Full non-functional cycle on an isolated JMeter environment and seeded load database: thread groups driving high-volume login, read and approve/deny traffic into the manager app, with closure analysis isolating password hashing as the throughput bottleneck. APACHE JMETER · SEEDED LOAD DATABASE · MANAGER API

OWNED

CONTRIBUTED

HELD IN ANOTHER LANE

Six-phase STLC coverage read left to right; each column is a phase, each row an owner.

93% / 86%

Branch coverage — Python employee API and Java manager API

4 defects

Logged, root-caused, fixed and locked behind regression tests

3 load plans

Login, read and write traffic on an isolated 5,000-record database

1 dashboard

Allure unifies unit, contract, E2E and coverage evidence